PROJECT REPORT

INTRODUCTION

This project is regarding Home Automation which was prototyped during the summer of 2021 and refined during the summer of 2023, if you are a student of science, you might find it interesting to go through and learn some new things about IoT (Internet of Things), Cloud Computing and Microcontrollers.

PURPOSE

The very reason for this project was to automate the home appliances (like: Lights, Fans, Air Conditioners. Etc.) and to know the status of every device/appliance right on our mobile devices or laptops or any other device which can access the internet.

APPARATUS

This project required many things from different points in time, here is a list of all the things used during the entirety of this project:

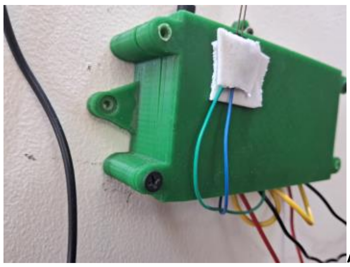
1. ESP8266:

It is a Wi-Fi microchip which means it can connect to any Wi-Fi router/Hotspot within range and I/O devices can exchange relevant signals over the internet. NodeMCU ESP8266 Wi-Fi Module is used in this project. Click here for the link of the same.

2. 4-Channel Relay Module:

 Relay is a device which can connect two wires on command basically acting as a switch, a switch that can be electronically triggered through a digital 0 or 1 signal coming from the microcontroller, here up to 4 appliances can be controlled. Click here.

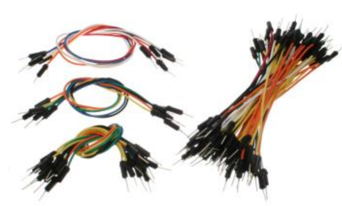
3. 3D Printed Case:

A case was designed using Blender with the intention of holding all the components including the Microcontroller, Relay Module, Wires. Etc. Unfortunately, the data of the same was wiped out of the system for some unforeseen circumstances but a model closest to the same can be found on Thingiverse. Click here.

4. Adapter (with Type B Cable):

An adapter is used to power the ESP and hence the Relay Module. It is important to note that ESP8266 is not 5V tolerant and giving it a direct supply might fry it, however in this project an adapter with almost 4 ~ 4.5V supply was used whereas the security limit (Voltage) for ESP8266 is 3.3V so it may be inferred as it has some tolerance.

5. Wires:

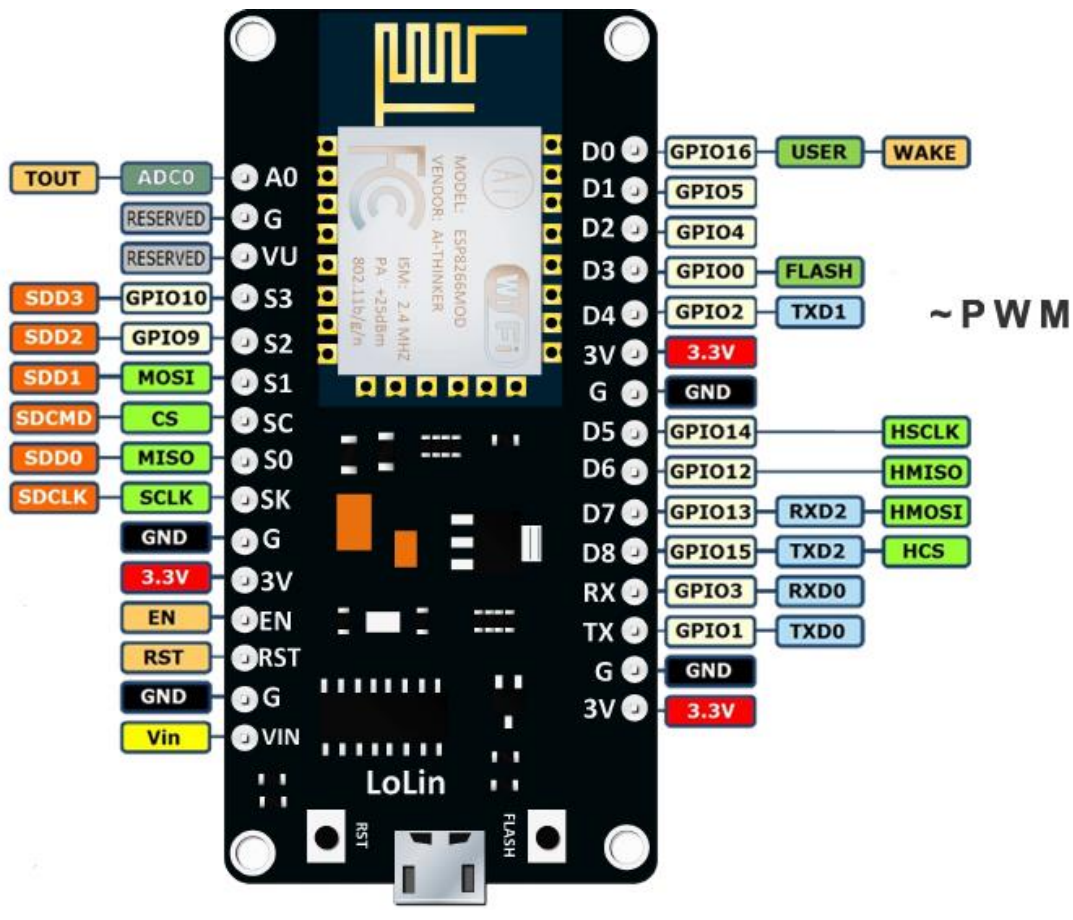
 A set of jumper wires were used to make the connections across the relevant pins of the Microcontroller and the Relay Module. Click here for jumper wires. Also, a set of silver wires were used to connect the retro switches to the Relay Modules.

6. IR (Infrared) LED:

An IR (Infrared) LED was used to turn ON and OFF the Air Conditioner.

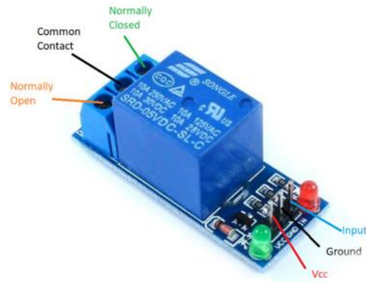
CIRCUITRY

To understand the circuitry of this project, firstly let us see the pinout diagram of ESP8266:



In this project GPIO (General Purpose Input/Output) Pins 2,5,12,13,14 are used, along with couple of 3.3V power supply and GND ground pins to provide power supply to the Relay Module.

Relay Circuitry:



Relay is another important circuitry to be discussed in this project. It contains three inlets for wires which are Normally Closed (NC), Normally Opened (NO) and Common Contact (Common). Initially when there is power supply and no external signal on the input pin, Inlets Common and NC are connected, and NO is open. But, as soon as there is an external signal on the input pin of the relay, Common and NO are connected, and NC is open. This makes the relay act as a CloudSwitch for our project.

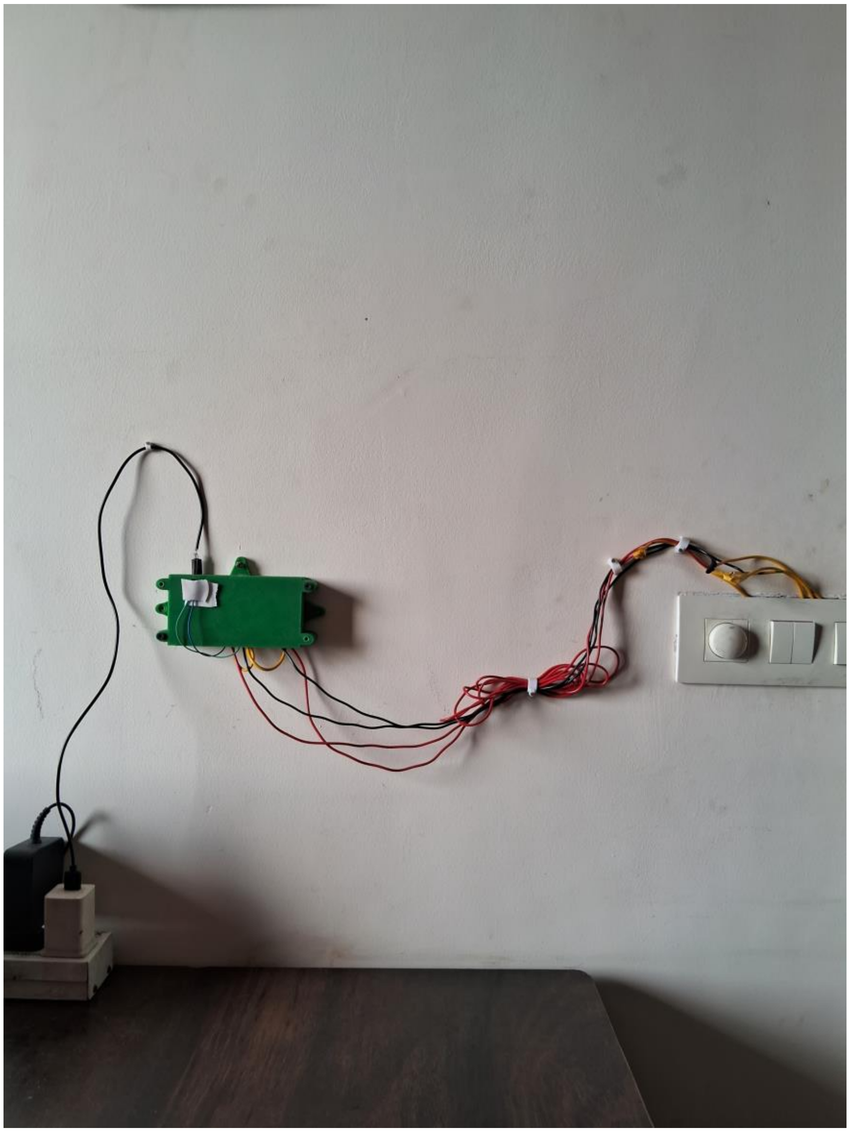


Image of the Room Automation Project

Note*   LED is wired up with ESP8266 using GPIO and GND pin.

Code Snippet

Arduino IoT Cloud was used to generate control signals to operate the appliances/Things!

Here are some code snippets for this project:

Cloud Variables

ADD

	Name	Last Value	Last Update
☐	**AC** CloudSwitch aC;	false	02 Aug 2023 13:38:48
☐	**Balcony_LED** CloudSwitch balcony_LED;	false	02 Aug 2023 13:38:33
☐	**Fan** CloudSwitch fan;	false	03 Aug 2023 21:47:19
☐	**White_LED** CloudSwitch white_LED;	true	02 Aug 2023 13:38:29
☐	**Yellow_LED** CloudSwitch yellow_LED;	false	02 Aug 2023 13:38:34

Living_Room_mar27a.in | ReadMe.adoc | thingProperties.h | Secret

```
/*
  The following variables are automatically generated and updated when changes are made to the Thing

  CloudSwitch balcony_LED;
  CloudSwitch fan;
  CloudSwitch white_LED;
  CloudSwitch yellow_LED;
*/

#include "thingProperties.h"
#include <Arduino.h>
#include <IRremoteESP8266.h>
#include <IRsend.h>

const uint16_t kIrLed = 4;

int i = 3;
int temp = i;

IRsend irsend(kIrLed);
```

```
IRsend irsend(kIrLed);

uint16_t rawDataON[211] = {3298, 3342,  400, 1268,  376, 446,  374, 1272,  402, 1268,  376,
                           444,  378, 446,  432, 1242,  404, 416,  378, 1270,  430, 392,  432, 1240,  376, 446,  432,
                           1214,  434, 1238,  404, 1242,  434, 416,  404, 416,  406, 1240,  432, 392,  432, 414,  406,
                           1240,  434, 1210,  436, 414,  404, 1240,  432, 1216,  432, 416,  404, 416,  432, 1214,  430,
                           416,  406, 418,  402, 418,  434, 1210,  434, 1240,  402, 1242,  430, 1240,  406, 1238,  434,
                           390,  432, 414,  404, 418,  406, 414,  434, 392,  430, 1240,  404, 1242,  432, 388,  432,
                           392,  430, 416,  406, 1238,  432, 388,  432, 1240,  406, 1240,  432, 1240,  406, 1240,
                           430, 1212,  432, 416,  406, 416,  432, 390,  434, 388,  434, 416,  404, 418,  402, 418,
                           434, 390,  434, 390,  430, 420,  404, 416,  402, 420,  432, 390,  432, 418,  404, 418,  402,
                           420,  430, 390,  432, 390,  434, 416,  404, 418,  404, 418,  430, 392,  434, 392,  430, 418,
                           404, 418,  434, 390,  430, 392,  432, 418,  404, 420,  374, 446,  432, 392,  432, 390,  430,
                           418,  402, 420,  404, 414,  434, 390,  432, 416,  406, 1240,  434, 1214,  430, 416,  404,
                           420,  404, 416,  432, 394,  430, 390,  432, 1242,  402, 1242,  430, 420,  404, 418,  402,
                           1242,  430, 480,  344, 416,  404
                          };  // HITACHI_AC1
```

```
uint16_t rawDataOFF[211] = {3266, 3346,  396, 1278,  368, 452,  370, 1274,  396, 1276,  372,
                            450,  372, 450,  396, 1278,  368, 452,  372, 1278,  392, 430,  392, 1252,  394, 428,  446,
                            1198,  450, 1222,  424, 1222,  448, 400,  424, 398,  366, 1280,  394, 428,  424, 452,  340,
                            1302,  398, 1248,  400, 452,  372, 1270,  402, 1246,  398, 448,  376, 448,  402, 1244,  400,
                            446,  372, 452,  370, 450,  400, 1246,  400, 1272,  374, 1272,  402, 1270,  374, 1274,  398,
                            424,  400, 452,  368, 452,  370, 452,  398, 424,  398, 1272,  374, 1272,  398, 428,  394,
                            424,  398, 452,  368, 1274,  402, 426,  396, 1272,  372, 1270,  398, 1274,  386, 1258,  400,
                            1246,  400, 450,  396, 426,  396, 424,  400, 420,  402, 450,  370, 450,  372, 450,  400,
                            426,  396, 424,  398, 450,  370, 452,  396, 424,  400, 422,  424, 426,  396, 428,  370, 448,
                            426, 396,  400, 426,  396, 450,  370, 452,  374, 450,  400, 422,  424, 398,  400, 448,  398,
                            424,  402, 420,  398, 430,  396, 448,  398, 428,  396, 424,  416, 406,  398, 424,  426, 426,
                            368, 452,  396, 424,  398, 424,  418, 430,  370, 452,  372, 1276,  396, 424,  426, 422,  372,
                            452,  398, 424,  400, 422,  424, 1248,  372, 452,  396, 428,  398, 420,  400, 1276,  370,
                            448,  424, 404,  418
                           };
```

```
int yellow = 2;
int white = 5;
int fan_ = 14;
int balcony = 12;
int tube = 13;

void setup() {
  pinMode(yellow, OUTPUT);
  pinMode(white, OUTPUT);
  pinMode(fan_, OUTPUT);
  pinMode(balcony, OUTPUT);
  pinMode(tube, OUTPUT);
  // Initialize serial and wait for port to open:
  Serial.begin(9600);
  // This delay gives the chance to wait for a Serial Monitor without blocking if none is found
  delay(1500);

  // Defined in thingProperties.h
  initProperties();
```

```
  initProperties();

  // Connect to Arduino IoT Cloud
  ArduinoCloud.begin(ArduinoIoTPreferredConnection);

  /*
     The following function allows you to obtain more information
     related to the state of network and IoT Cloud connection and errors
     the higher number the more granular information you'll get.
     The default is 0 (only errors).
     Maximum is 4
  */
  setDebugMessageLevel(2);
  ArduinoCloud.printDebugInfo();

  irsend.begin();
#if ESP8266
  Serial.begin(115200, SERIAL_8N1, SERIAL_TX_ONLY);
#else  // ESP8266
  Serial.begin(115200, SERIAL_8N1);
#endif  // ESP8266
```

```
  Serial.begin(115200, SERIAL_8N1);
#endif  // ESP8266
}

void loop() {
  ArduinoCloud.update();

}

/*
  Since YellowLED is READ_WRITE variable, onYellowLEDChange() is
  executed every time a new value is received from IoT Cloud.
*/
void onYellowLEDChange()  {
  if (yellow_LED)
    digitalWrite(yellow, LOW);
  else
    digitalWrite(yellow, HIGH);
}

```

```
void onBalconyLEDChange()  {
  if (balcony_LED)
    digitalWrite(balcony, LOW);
  else
    digitalWrite(balcony, HIGH);
}

/*
  Since Fan is READ_WRITE variable, onFanChange() is
  executed every time a new value is received from IoT Cloud.
*/
void onFanChange()  {
  if (fan)
    digitalWrite(fan_, LOW);
  else
    digitalWrite(fan_, HIGH);
}

```

```

/*
  Since WhiteLED is READ_WRITE variable, onWhiteLEDChange() is
  executed every time a new value is received from IoT Cloud.
*/
void onWhiteLEDChange()  {
  if (white_LED)
    digitalWrite(white, LOW);
  else
    digitalWrite(white, HIGH);
}
```

```
void onACChange()  {
  if (aC)
  {
    while (i)
    {
      irsend.sendRaw(rawDataON, 211, 38);  // Send a raw data capture at 38kHz.
      delay(500);
      i = i - 1;
    }
    i = temp;
  }
  else
  {
    while (i)
    {
      irsend.sendRaw(rawDataOFF, 211, 38);  // Send a raw data capture at 38kHz.
      delay(500);
      i = i - 1;
    }
    i = temp;
  }
}
```

Challenges

On the boot, some pins of the ESP8266 are HIGH which turns ON the appliances connected to them, and it becomes difficult to handle power cuts due to this fact.

Handling the appliances/Things with the pins which stay LOW on the boot or using a UPS (uninterrupted Power Supply) could be proposed.

References

Arduino IRRemote Library by Ken Sherriff : [GitHub - Arduino-IRremote/Arduino-IRremote: Infrared remote library for Arduino: send and receive infrared signals with multiple protocols]